



UNIVERSIDADE FEDERAL DE LAVRAS
ICTIN – INSTITUTO DE TECNOLOGIA E INOVAÇÃO

**PROJETO 2 – IMPLEMENTAÇÃO DE GRAFOS
COM ADJACENCIA EM JAVA**

MARIA JULIA PEDROSO PIMENTA
JUKA FRANCIS PEREIRA GONÇALVES

SÃO SEBASTIÃO DO PARAÍSO
05/2026

Sumário

1	Introdução	2
2	Grafos Não Direcionados	2
2.1	Representações Computacionais	3
3	Lista de Adjacência	3
3.1	Implementação em Java	3
4	Busca em Profundidade (DFS)	4
4.1	O Array de Visitados	5
4.2	Padrão Público/Privado em Java	5
5	Busca em Largura (BFS)	6
5.1	Momento de Marcar como Visitado	6
6	Diferença Conceitual entre DFS e BFS	7
7	Por que BFS Encontra o Menor Caminho em Grafos Não Ponderados	7
8	Componentes Conexas	8
9	Grafo de Teste	9
10	Análise de Complexidade	10
11	Conclusão	10
	Referências	12

1 Introdução

Grafos são estruturas de dados fundamentais na Ciência da Computação, com aplicações que vão desde redes sociais e sistemas de navegação até compiladores e análise de redes de computadores. A capacidade de modelar relações entre entidades por meio de vértices e arestas torna os grafos uma ferramenta indispensável para a resolução de problemas computacionais complexos.

Este relatório documenta o desenvolvimento do Projeto 02 da disciplina GCT153 – Estruturas de Dados II, cujo objetivo central é a implementação de um grafo não direcionado utilizando lista de adjacência como estrutura de representação, bem como o desenvolvimento de algoritmos clássicos de percurso: Busca em Profundidade (DFS – *Depth-First Search*), Busca em Largura (BFS – *Breadth-First Search*) e identificação de Componentes Conexas.

A implementação foi realizada na linguagem Java, com foco na aplicação de princípios de Programação Orientada a Objetos (POO), modularidade e boas práticas de engenharia de software. O projeto parte de um grafo de teste predefinido com dez vértices e sete arestas, incluindo um vértice isolado e um ciclo, o que permite validar os algoritmos em diferentes cenários estruturais.

2 Grafos Não Direcionados

Um grafo G é definido formalmente como um par ordenado $G = (V, E)$, onde V é um conjunto finito de vértices (também chamados de nós) e E é um conjunto de arestas, onde cada aresta é um par não ordenado de vértices $\{u, v\}$, com $u, v \in V$.

Em um grafo **não direcionado**, as arestas não possuem orientação. Isso significa que se existe uma aresta entre os vértices u e v , a relação é simétrica: pode-se navegar de u para v e de v para u com igual validade. Formalmente, $\{u, v\} = \{v, u\}$.

No contexto deste projeto, algumas propriedades estruturais do grafo de teste merecem atenção:

- **Vértice isolado:** o vértice 9 não possui nenhuma aresta incidente, ou seja, seu grau é zero. Algoritmos de percurso iniciados em outros vértices não o alcançam;
- **Ciclo:** os vértices 6, 7 e 8 formam um ciclo ($6 \rightarrow 7 \rightarrow 8 \rightarrow 6$). Algoritmos que não controlam vértices visitados entrariam em loop infinito nessa estrutura;
- **Cadeia linear:** os vértices 0, 1, 2 e 3 formam uma cadeia simples sem ramificações.

2.1 Representações Computacionais

Existem duas formas principais de representar um grafo em memória:

Critério	Matriz de Adjacência	Lista de Adjacência
Espaço em memória	$O(V^2)$	$O(V + E)$
Verificar adjacência	$O(1)$	$O(\text{grau}(v))$
Listar vizinhos	$O(V)$	$O(\text{grau}(v))$
Ideal para	Grafos densos	Grafos esparsos

Tabela 1: Comparação entre representações de grafos

Para este projeto, a lista de adjacência é a escolha mais eficiente: com apenas 7 arestas e 10 vértices, uma matriz de adjacência alocaria 100 células, das quais a maioria representaria ausências. A lista de adjacência aloca apenas o necessário.

3 Lista de Adjacência

A lista de adjacência é uma estrutura de dados que representa um grafo associando a cada vértice uma lista de seus vizinhos diretos. Para um grafo com n vértices, mantém-se um array de n listas, onde a lista de índice v contém todos os vértices adjacentes a v .

Após a inserção de todas as arestas do grafo de teste, a lista de adjacência resultante é:

```
Vertice 0 -> [1]
Vertice 1 -> [0, 2]
Vertice 2 -> [1, 3]
Vertice 3 -> [2]
Vertice 4 -> [5]
Vertice 5 -> [4]
Vertice 6 -> [7, 8]
Vertice 7 -> [6, 8]
Vertice 8 -> [7, 6]
Vertice 9 -> []
```

Listing 1: Lista de adjacência do grafo de teste

Observa-se que, por ser um grafo não direcionado, cada aresta é registrada duas vezes: a aresta $\{0, 1\}$ aparece na lista do vértice 0 (como vizinho 1) e na lista do vértice 1 (como vizinho 0).

3.1 Implementação em Java

Em Java, a lista de adjacência foi implementada como um array de `ArrayList<Integer>`, onde cada posição do array corresponde a um vértice:

```
1 private ArrayList<Integer>[] listaAdj;  
2  
3 @SuppressWarnings("unchecked")  
4 public Grafos(int n) {  
5     qtdVertices = n;  
6     listaAdj = new ArrayList[n];  
7     for (int i = 0; i < n; i++) {  
8         listaAdj[i] = new ArrayList<>();  
9     }  
10 }
```

Listing 2: Declaração e inicialização da lista de adjacência

A anotação `@SuppressWarnings("unchecked")` é necessária devido a uma limitação histórica do Java: a linguagem não permite a criação direta de arrays genéricos parametrizados (*type erasure*). Em tempo de execução, o tipo genérico `<Integer>` é apagado, o que gera um aviso do compilador, mas não compromete o funcionamento do código.

A inserção de arestas registra a conexão nos dois sentidos, garantindo a simetria do grafo não direcionado:

```
1 public void inserirAresta(int origem, int destino) {  
2     listaAdj[origem].add(destino);  
3     listaAdj[destino].add(origem);  
4 }
```

Listing 3: Inserção de aresta não direcionada

4 Busca em Profundidade (DFS)

A Busca em Profundidade (*Depth-First Search* – DFS) é um algoritmo de percurso em grafos que explora cada caminho até o seu limite antes de retroceder e tentar outros caminhos. A ideia central é: a partir de um vértice de origem, avançar o mais fundo possível antes de recuar (*backtracking*).

O algoritmo utiliza implicitamente uma pilha, seja por meio de chamadas recursivas (que utilizam a pilha de execução do sistema) ou explicitamente com uma estrutura de pilha.

Aplicando DFS a partir do vértice 0 no grafo de teste:

1. Visita o vértice 0. Marca como visitado. Empilha chamada.
2. Avança para o vizinho 1. Marca como visitado. Empilha chamada.
3. Avança para o vizinho 2. Marca como visitado. Empilha chamada.

4. Avança para o vizinho 3. Marca como visitado. Sem vizinhos não visitados.
5. *Backtrack* para 2. Sem outros vizinhos não visitados.
6. *Backtrack* para 1. Sem outros vizinhos não visitados.
7. *Backtrack* para 0. Sem outros vizinhos não visitados. Encerra.

Resultado: 0 1 2 3

4.1 O Array de Visitados

O array `boolean[] visitado` é fundamental para evitar ciclos infinitos. No ciclo $6 \rightarrow 7 \rightarrow 8 \rightarrow 6$, sem controle de visitados, o algoritmo navegaria indefinidamente. O array garante que cada vértice seja processado exatamente uma vez.

4.2 Padrão Público/Privado em Java

A implementação adota um padrão clássico de POO: um método público prepara o estado inicial, e um método privado executa a recursão. Isso evita que o array `visitado` seja recriado a cada chamada recursiva – o que zeraria o histórico de exploração.

```
1 public void dfs(int origem) {
2     boolean[] visitado = new boolean[qtdVertices];
3     System.out.print("Ordem DFS: ");
4     dfsAuxiliar(origem, visitado);
5     System.out.println();
6 }
7
8 private void dfsAuxiliar(int v, boolean[] visitado) {
9     visitado[v] = true;
10    System.out.print(v + " ");
11    for (Integer vizinho : listaAdj[v]) {
12        if (!visitado[vizinho]) {
13            dfsAuxiliar(vizinho, visitado);
14        }
15    }
16 }
```

Listing 4: Implementação do DFS com padrão público/privado

O array `visitado` é criado uma única vez no método público e passado por referência para todas as chamadas recursivas – funcionando como memória compartilhada da exploração.

5 Busca em Largura (BFS)

A Busca em Largura (*Breadth-First Search* – BFS) é um algoritmo de percurso que explora o grafo nível por nível: primeiro visita todos os vértices a distância 1 do ponto de origem, depois todos a distância 2, e assim sucessivamente. A estrutura de dados utilizada é a fila (FIFO – *First In, First Out*).

Aplicando BFS a partir do vértice 0:

1. Enfileira o vértice 0. Marca como visitado.
2. Desenfileira 0. Visita. Enfileira vizinhos não visitados: [1].
3. Desenfileira 1. Visita. Enfileira vizinhos não visitados: [2].
4. Desenfileira 2. Visita. Enfileira vizinhos não visitados: [3].
5. Desenfileira 3. Visita. Sem vizinhos não visitados. Fila vazia. Encerra.

Resultado: 0 1 2 3

5.1 Momento de Marcar como Visitado

Um detalhe crítico na implementação do BFS é **marcar o vértice como visitado no momento em que é enfileirado**, não quando é desenfileirado. Caso contrário, em grafos com múltiplas arestas chegando ao mesmo vértice, ele poderia ser enfileirado mais de uma vez, gerando processamento duplicado e resultados incorretos.

```
1 public void bfs(int origem) {
2     boolean[] visitado = new boolean[qtdVertices];
3     Queue<Integer> fila = new LinkedList<>();
4     visitado[origem] = true;    // marca AO ENFILEIRAR
5     fila.offer(origem);
6     System.out.print("Ordem do BFS: ");
7     while (!fila.isEmpty()) {
8         int v = fila.poll();
9         System.out.print(v + " ");
10        for (Integer vizinho : listaAdj[v]) {
11            if (!visitado[vizinho]) {
12                visitado[vizinho] = true;    // marca AO ENFILEIRAR
13                fila.offer(vizinho);
14            }
15        }
16    }
17    System.out.println();
18 }
```

Listing 5: Implementação do BFS com fila

6 Diferença Conceitual entre DFS e BFS

Embora DFS e BFS compartilhem a mesma complexidade assintótica $O(V + E)$ e ambos visitem todos os vértices alcançáveis a partir de uma origem, diferem fundamentalmente na ordem de exploração e na estrutura de dados utilizada.

Critério	DFS	BFS
Estrutura interna	Pilha (recursão)	Fila
Estratégia	Vai fundo antes de voltar	Explora por níveis
Ordem de visita	Profundidade primeiro	Largura primeiro
Caminho mais curto	Não garante	Garante (não ponderado)
Uso de memória (pior caso)	$O(V)$ – pilha de recursão	$O(V)$ – fila
Adequado para	Detecção de ciclos, topologia	Menor caminho, níveis

Tabela 2: Comparação direta entre DFS e BFS

No grafo de teste, DFS e BFS produziram a mesma ordem de visita (0 1 2 3) porque a estrutura {0-1-2-3} é uma cadeia linear sem ramificações. Em grafos com ramificações, as ordens divergem: o DFS aprofundaria um ramo completamente antes de explorar o outro, enquanto o BFS visitaria todos os vértices do mesmo nível antes de avançar.

7 Por que BFS Encontra o Menor Caminho em Grafos Não Ponderados

Em um grafo não ponderado, o menor caminho entre dois vértices é aquele com o menor número de arestas. O BFS garante encontrar esse caminho por uma propriedade direta de seu funcionamento: a exploração por níveis.

Quando o BFS parte de uma origem s , ele visita primeiro todos os vértices a distância 1 (vizinhos diretos), depois todos a distância 2, depois distância 3, e assim por diante. Isso cria uma fronteira de exploração que avança uniformemente.

A consequência direta é: o primeiro momento em que o BFS alcança um vértice destino d , o caminho percorrido até ele é necessariamente o mais curto em número de arestas. Qualquer caminho alternativo mais curto já teria sido descoberto em uma iteração anterior, pois a fila garante ordem crescente de distância. Formalmente, ao ser desenfileirado, todo vértice v possui distância $d(s, v)$ já estabelecida como mínima. Isso não ocorre no DFS, que pode chegar a um vértice por um caminho longo antes de descobrir um caminho mais curto.

Importante: essa propriedade vale apenas para grafos não ponderados. Em grafos com pesos nas arestas, o algoritmo adequado para encontrar o caminho mínimo é o algoritmo de Dijkstra, que generaliza o BFS considerando os custos das arestas.

8 Componentes Conexas

Uma componente conexa de um grafo não direcionado é um subconjunto maximal de vértices $C \subseteq V$ tal que existe um caminho entre qualquer par de vértices $u, v \in C$, e não existe aresta conectando C a nenhum vértice fora de C . Em outras palavras: é um grupo de vértices que conseguem se alcançar mutuamente, completamente isolado do restante do grafo.

O grafo de teste possui quatro componentes conexas distintas:

Componente	Vértices	Estrutura
1	{0, 1, 2, 3}	Cadeia linear
2	{4, 5}	Par simples
3	{6, 7, 8}	Ciclo
4	{9}	Vértice isolado

Tabela 3: Componentes conexas do grafo de teste

A identificação de componentes conexas é obtida por uma extensão natural do DFS: percorre-se todos os vértices do grafo e, para cada vértice ainda não visitado, executa-se uma nova busca em profundidade. Cada nova busca cobre exatamente uma componente conexa.

O ponto crítico é que o array `visitado` é compartilhado entre todas as chamadas, permitindo que o algoritmo saiba quais vértices já foram cobertos por componentes anteriores.

```

1 public void componentesConexas() {
2     boolean[] visitado = new boolean[qtdVertices];
3     int contador = 0;
4     for (int v = 0; v < qtdVertices; v++) {
5         if (!visitado[v]) {
6             contador++;
7             System.out.print("Componente " + contador + ": ");
8             dfsAuxiliar(v, visitado);
9             System.out.println();
10        }
11    }
12    System.out.println("Total de componentes conexas: " + contador);
13 }

```

Listing 6: Identificação de componentes conexas

A reutilização do método `dfsAuxiliar()` neste contexto exemplifica um princípio importante de engenharia de software: a reusabilidade de código. O mesmo método privado que serve ao DFS convencional é aproveitado na identificação de componentes, evitando duplicação de lógica.

9 Grafo de Teste

O grafo utilizado nos testes possui as seguintes características:

- **Vértices:** 0 a 9 (total de 10 vértices)
- **Arestas:** 0-1, 1-2, 2-3, 4-5, 6-7, 7-8, 8-6 (total de 7 arestas)
- **Tipo:** não direcionado, não ponderado
- **Vértice isolado:** 9 (grau zero)

Todos os resultados estão de acordo com o esperado, validando a corretude das implementações. Saída Completa da Execução:

```
Vertice 0-> [1]
Vertice 1-> [0, 2]
Vertice 2-> [1, 3]
Vertice 3-> [2]
Vertice 4-> [5]
Vertice 5-> [4]
Vertice 6-> [7, 8]
Vertice 7-> [6, 8]
Vertice 8-> [7, 6]
Vertice 9-> []

Ordem DFS: 0 1 2 3
Ordem DFS: 4 5
Ordem DFS: 6 7 8
Ordem DFS: 9

Ordem do BFS: 0 1 2 3
Ordem do BFS: 4 5
Ordem do BFS: 6 7 8
Ordem do BFS: 9

Componente 1: 0 1 2 3
Componente 2: 4 5
Componente 3: 6 7 8
Componente 4: 9
Total de componentes conexas: 4
```

Listing 7: Saída completa do programa

10 Análise de Complexidade

Utiliza-se a notação O (Big-O) para expressar o limite superior de crescimento do tempo de execução e uso de memória. As variáveis adotadas são: V para o número de vértices e E para o número de arestas.

Operação	Tempo	Espaço Auxiliar
Construção do grafo	$O(V)$	$O(V + E)$
Inserção de aresta	$O(1)$	$O(1)$
Exibição do grafo	$O(V + E)$	$O(1)$
DFS	$O(V + E)$	$O(V)$
BFS	$O(V + E)$	$O(V)$
Componentes Conexas	$O(V + E)$	$O(V)$

Tabela 4: Complexidade de tempo e espaço dos algoritmos

DFS e BFS – $O(V + E)$: ambos visitam cada vértice exatamente uma vez ($O(V)$) e percorrem cada lista de adjacência uma vez ($O(E)$ no total). O array `visitado` garante que nenhum vértice seja reprocessado.

Espaço $O(V)$: o array `visitado` ocupa $O(V)$ em todos os algoritmos. No DFS recursivo, a pilha de chamadas pode crescer até $O(V)$ no pior caso (um caminho linear com todos os vértices). No BFS, a fila pode conter até $O(V)$ elementos simultaneamente.

Componentes Conexas – $O(V + E)$: o loop externo visita cada vértice uma vez ($O(V)$), e o `dfsAuxiliar` interno percorre cada aresta no total uma única vez ($O(E)$). O array compartilhado impede reprocessamento.

Espaço total da estrutura – $O(V + E)$: a lista de adjacência armazena V listas com um total de $2E$ entradas (cada aresta registrada duas vezes por ser não direcionado).

11 Conclusão

Este projeto permitiu consolidar na prática os conceitos teóricos de grafos estudados na disciplina. A implementação em Java com Programação Orientada a Objetos evidenciou a importância do design de software antes da codificação: decisões como a separação entre métodos públicos e privados, o compartilhamento do array `visitado` entre chamadas recursivas e a reutilização do `dfsAuxiliar` no algoritmo de componentes conexas impactam diretamente a corretude e a qualidade do código.

A escolha da lista de adjacência como estrutura de representação mostrou-se adequada para grafos esparsos como o utilizado neste projeto, oferecendo eficiência de memória superior à matriz de adjacência. Os algoritmos DFS e BFS, embora possuam a mesma complexidade assintótica $O(V + E)$, apresentam comportamentos distintos que os tornam adequados para problemas diferentes: o DFS é naturalmente aplicado a detecção de ciclos

e ordenação topológica, enquanto o BFS é a escolha ideal quando se busca o caminho com menor número de arestas.

O grafo de teste, cuidadosamente escolhido com quatro componentes conexas distintas (cadeia linear, um par, um ciclo e um vértice isolado) permitindo validar os algoritmos em diferentes estruturas, confirmando a robustez da implementação.

Referências

CORMEN, Thomas H. et al. **Introdução a Algoritmos**. 3. ed. Rio de Janeiro: Elsevier, 2012.

SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Upper Saddle River: Addison-Wesley, 2011.

GOODRICH, Michael T.; TAMASSIA, Roberto. **Estruturas de Dados e Algoritmos em Java**. 5. ed. Porto Alegre: Bookman, 2013.

SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. **Estruturas de Dados e seus Algoritmos**. 3. ed. Rio de Janeiro: LTC, 2010.

ORACLE. *Java Platform, Standard Edition 17 API Specification*. Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/>. Acesso em: maio 2026.